

Práctica 2: Syscalls - marcapasos VVI

Práctica 2 — Syscalls (marcapasos VVI)

Capa de llamadas al sistema sobre FreeRTOS/ESP32. La lógica médica no toca GPIO, ISR ni el Task WDT: todo pasa por un despachador KernelService de prioridad máxima.

Cómo simular en Wokwi

1. Abrir [un proyecto ESP32 nuevo](#).
2. Pegar `diagram.json` y `sketch.ino`.
3. Añadir los demás archivos al proyecto (`bootstrap.h`, `bootstrap.cpp`, `syscalls.h`, `syscalls.cpp`).
4. Iniciar la simulación (monitor serie a 115200).

Pin	Rol	Hardware Wokwi
GPIO 4	Sensado ventricular (onda R)	Botón amarillo a GND
GPIO 5	Pulso de estimulación	LED rojo + 220 Ω

2. Investigación previa

1. Modos de privilegio

En un SO clásico, **modo usuario** ejecuta la aplicación con un subconjunto restringido de instrucciones y sin acceso a registros de I/O ni a tablas del kernel. **Modo supervisor (kernel)** puede programar MMU, GPIO, temporizadores e interrupciones.

Que el código médico escriba pines o registros a mano es peligroso porque:

- un bug (puntero colgante, ancho de pulso erróneo) actúa *directamente* sobre el tejido simulado;
- no hay un punto único de validación ni de auditoría;
- dos tareas pueden intercalar escrituras y dejar el pin en un estado inconsistente;
- se pierde el aislamiento de fallos: un error de aplicación se convierte en un fallo de hardware.

En el ESP32 no hay un cambio de anillo de CPU tan estricto como en un MMU de escritorio, así que **emulamos** esa frontera con una cola de solicitudes y un hilo supervisor. La aplicación solo ve `syscalls.h`.

2. Qué es una Syscall

Una **syscall** es la vía controlada por la que una tarea de aplicación pide un servicio privilegiado. No es una llamada C normal:

	Función C	Syscall
Destino	Otra función en el mismo espacio	Despachador de kernel
Cambio de contexto	Call/ret, misma pila	Bloqueo + cambio de tarea (o trap de hardware)
Validación	La que ponga el programador	Obligatoria en el supervisor
Retorno	Registro / pila	Semáforo de completado + campo result

Aquí el “trap” se emula así: la API pública rellena un `syscall_req_t`, lo encola y se duerme en un semáforo binario. `KernelService` atiende, escribe el resultado y despierta al llamante.

3. ISR — qué está prohibido

Una **ISR** (Rutina de Servicio de Interrupción) corre en respuesta a un evento de hardware, fuera del planificador, con las interrupciones a menudo enmascaradas.

Prohibido dentro de la ISR de sensado:

1. **delay() / vTaskDelay() / esperas largas.** La ISR no es una tarea: bloquearla congela *todas* las interrupciones de igual o menor prioridad y rompe el determinismo (el pulso de 500 μ s y el WDT dejarían de atenderse).
2. **Serial.printf / malloc / mutexes ordinarios.** El puerto serie y el heap no son reentrantes desde ISR; un `printf` puede tardar milisegundos y un `malloc` puede bloquearse. FreeRTOS solo permite primitivas `FromISR`.

La ISR de este firmware solo hace `xSemaphoreGiveFromISR` y, si hace falta, un `portYIELD_FROM_ISR`.

4. Cola y semáforo binario

- **Cola (Queue):** buffer de mensajes con exclusión mutua. Aquí transporta *punteros* a `syscall_req_t` desde la aplicación hacia `KernelService`.
- **Semáforo binario:** bandera 0/1. Se usa en dos sitios:
 - `sense_sem`: la ISR lo libera; el kernel lo toma con timeout (desbloqueo de `sys_wait_sensing`).
 - `done_sem`: el kernel lo libera cuando terminó; la API pública se bloquea ahí a la espera del resultado.

Sin estas primitivas, la ISR tendría que “avisar” a la tarea médica tocando variables globales y arriesgando condiciones de carrera. `GiveFromISR` + `Take` es el patrón ISR → tarea de FreeRTOS.

5. Lógica VVI

- **Intervalo de escape (Tescape = 1000 ms, 60 ppm):** si no hay despolarización intrínseca en ese tiempo, el marcapasos *debe* estimular. Es la frecuencia mínima garantizada.
 - **Periodo refractario ventricular (VRP = 250 ms):** tras un sensado o un pulso, se ignoran flancos. Evita tomar la onda T (o el rebote del propio pulso) como una nueva onda R (*oversensing*).
 - **Ventana de escucha** = Tescape – VRP = **750 ms**.
-

4.1 Análisis de requerimientos y diseño

Requerimientos funcionales

ID	Entrada	Salida / efecto	Comportamiento
SYS_GET_TIME	—	uint64_t μ s monotónico	esp_timer_get_time(), seguro en conurrencia (un solo despachador)
SYS_WAIT_SENSE	timeout ms	bool	Bloquea hasta onda R o timeout; notificación desde ISR
SYS_PACE_PULSE	ancho μ s	bool	Pulso atómico en GPIO 5 solo si 100– 2000 μ s
SYS_LOG_EVENT	const char *	telemetría	Copia al anillo circular, libera a la app y luego imprime por Serial (no bloquea la tarea médica); rechaza puntero nulo
SYS_KICK_WDT	—	reset del TWDT	Solo el hilo supervisor, tras recibir la petición de la app
Control VVI	—	ritmo 60 ppm	Escucha Tescape – tiempo ya consumido ($\approx$ 750 ms) – inhibe o estimula 500 μ s – duerme VRP 250 ms

Requerimientos no funcionales y restricciones

Tipo	Restricción
Latencia de pulso	$\leq 100 \mu\text{s}$ desde la solicitud hasta el flanco de subida
Latencia de sensado	notificación ISR $\rightarrow$ semáforo sin trabajo pesado (objetivo $\leq 50 \mu\text{s}$)
Privilegio	la app no llama <code>digitalWrite</code> / <code>attachInterrupt</code> / registros
Despacho	cola + <code>KernelService</code> a <code>configMAX_PRIORITIES - 1</code>
Fallos	parámetro inválido $\rightarrow$ anomalía en log, false, sin panic/reboot
WDT	timeout 3 s; lo alimenta únicamente el supervisor

Qué es público y qué es interno

syscalls.h (aplicación): `vvi_sys_init`, `sys_get_time_us`, `sys_wait_sensing`, `sys_pace_pulse`, `sys_log_event`, `sys_kick_watchdog`, `sys_sleep_ms`.

syscalls.cpp (kernel): pines, ISR, cola, semáforos, `syscall_req_t`, anillo de log, `kernel_service_task`, generación física del pulso.

bootstrap.*: POST de Flash/SRAM y un WDT ya inicializado (Práctica 1). No se suscribe el hilo de Arduino: el único vigilado es `KernelService`.

Por qué una cola y no una función compartida

Una función C compartida (`digitalWrite` desde la app) no serializa el acceso, no valida en un solo sitio y no puede imponer prioridad. La cola:

1. convierte cada pedido en un mensaje atómico;
2. lo atiende un solo hilo a prioridad máxima (el pulso no lo interrumpe `VVI_Task`);
3. deja un registro natural (el switch del despachador) para telemetría y rechazo.

`sys_sleep_ms` es la excepción: es `vTaskDelay` en espacio de usuario. Si el *kernel* durmiera 250 ms, el despachador no podría atender el WDT ni otra syscall.

Estructura genérica de solicitud

```
typedef struct {
    syscall_id_t id;      /* qué servicio */
    union { ... } args;   /* timeout / ancho / mensaje */
    union { ... } result; /* time_us / success */
    SemaphoreHandle_t done_sem; /* desbloquea al llamante */
} syscall_req_t;
```

La cola guarda **punteros**, no copias. Si se encolara la struct por valor (como en el fragmento de la guía), req.result se escribiría en la copia local del kernel y el llamante nunca vería el retorno.

Sincronización de retorno: **semáforo binario** (done_sem). Una cola de respuesta también serviría; el semáforo es más ligero cuando el resultado ya viaja dentro de la misma struct.

Flujo de sensado (ISR → syscall)

onda R → GPIO 4 falling → sensing_isr
 → xSemaphoreGiveFromISR(sense_sem)
 → KernelService (bloqueado en xSemaphoreTake)
 → req.result.success = true
 → xSemaphoreGive(done_sem)
 → VVI_Task despierta y inhibe el pulso

Antes de esperar, el kernel **drena** un token residual (xSemaphoreTake(..., 0)). Así un clic durante el VRP no se “guarda” para la siguiente ventana de escucha.

Validación de sys_pace_pulse

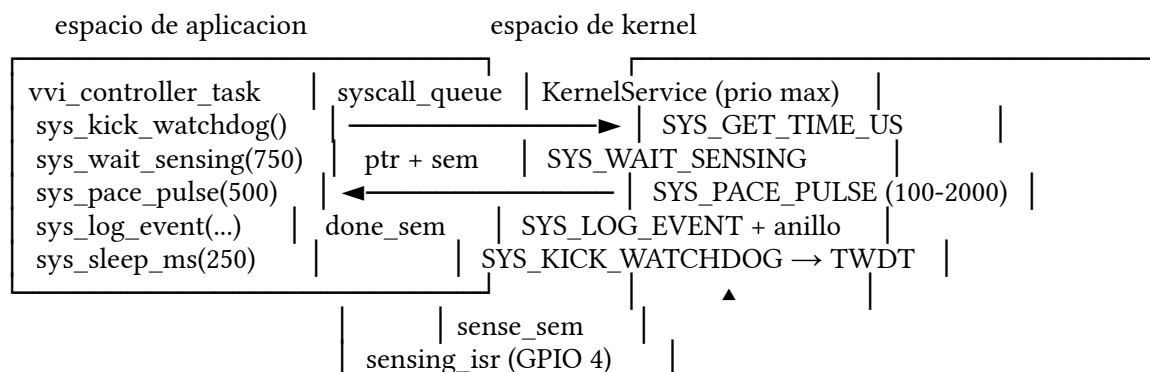
Ancho	Acción del kernel
< 100 µs o > 2000 µs	log [KERNEL ANOMALY], success = false, GPIO 5 intacto
100–2000 µs	HIGH → delayMicroseconds → LOW, success = true

No hay abort, esp_restart ni assert.

Watchdog

bootstrap_init configura el TWDT (3 s, panic). kernel_service_task es el único que hace esp_task_wdt_add. Solo se resetea al atender SYS_KICK_WATCHDOG, es decir, cuando la aplicación médica sigue viva y pide el servicio. Si VVI_Task se cuelga, deja de encolar kicks y el TWDT reinicia.

Arquitectura



pulso GPIO 5

Ciclo VVI: $\approx 750 \text{ ms escuchando} + (\text{pulso } 500 \mu\text{s o inhibición}) + 250 \text{ ms VRP} = 1000 \text{ ms}$ entre escapes.

La ventana de escucha no es una constante: la tarea toma `sys_get_time_us()` al inicio de cada ciclo y resta lo que ya se consumió desde el último evento (VRP, log, pulso). Así el overhead del `printf` o del pulso no se acumula y los escapes quedan a 1 000 000 μs exactos en las marcas `[KERNEL LOG @ ...us]`.

Limitación conocida

`SYS_WAIT_SENSING` bloquea al despachador hasta la onda R o el timeout (así lo pide la plantilla). Con una sola tarea médica es correcto; si hubiera más tareas de aplicación, ese servicio debería atenderse fuera del hilo `KernelService` (p. ej. entregando el semáforo directamente a la tarea solicitante) para no congelar el resto de syscalls.

4.3 Pruebas (escenarios clínicos)

Dejar `PACE_PULSE_US` en 500 salvo el escenario 4.

Escenario 1 — Estimulación por escape

Acción: arrancar y no tocar el botón.

Esperado: cada $\sim 1000 \text{ ms}$ aparece Escape agotado. Estimulando ventriculo.... El LED rojo destella $500 \mu\text{s}$. El WDT no reinicia. Restar marcas `[KERNEL LOG @ ...us]` de dos escapes consecutivos: la diferencia debe acercarse a 1 000 000 μs .

Escenario 2 — Inhibición por onda R

Acción: pulsar el botón $\sim 500 \text{ ms}$ después del último evento.

Esperado: Onda R detectada. Estimulación inhibida. El LED **no** enciende. El intervalo de escape se reinicia en ese instante.

Escenario 3 — Inmunidad en VRP

Acción: justo al ver un log de escape o de onda R, pulsar el botón en ráfaga.

Esperado: esos clics no generan Onda R detectada ni adelantan el ritmo. El kernel descarta el semáforo al abrir la siguiente ventana.

Escenario 4 — Validación defensiva

Acción: en `sketch.ino` poner `#define PACE_PULSE_US 3000` (o 50) y reiniciar.

Esperado: `[KERNEL ANOMALY] SYS_PACE_PULSE rechazado`, retorno falso, LED apagado. Restaurar 500 al terminar.

Evidencias

Las capturas de la carga del circuito y del servidor de compilación están en evidencias/. El monitor serie y las pruebas del botón siguen pendientes; ver ../VALIDACION_WOKWI.md. Si se usa hardware, añadir una foto del montaje (botón en GPIO 4, LED en GPIO 5).

Archivos

Archivo	Rol
diagram.json	Topología Wokwi (ESP32 + botón + LED)
bootstrap.h/.cpp	POST + WDT de la Práctica 1
syscalls.h	API pública
syscalls.cpp	Despachador, ISR, pulso, anillo de log, WDT
sketch.ino	Tarea médica VVI

Autoevaluación (guía)

Escala 1–5. Completar según la experiencia de cada integrante.

Afirmación	Valor
Los conceptos son relevantes para mi formación profesional	
El trabajo permitió comprender la teoría de clase	
Las actividades resultaron atractivas	
El enfoque práctico captó mi interés	
La complejidad fue adecuada	
La guía permitió trabajar sin bloqueos innecesarios	